

Untitled

August 19, 2026

```
[1]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

0.1 ETL Phase

```
[2]: # Importing data
spotify_origin = pd.read_csv("C:\\Users\\ihshan\\Data Analysis\\Most Streamed_
↳Artists on Spotify\\Data\\Most Streamed Artists on Spotify (17_07_2026) V1.1.
↳csv")

# Making copy of the dataframe to preserve the original copy
spotify = spotify_origin.copy()

# Checking df shape
spotify.shape
```

```
[2]: (500, 14)
```

```
[3]: # Previewing df

spotify_origin.head()
```

```
[3]:
```

	Artist Name	Sex	Country of Origin	Primary Language	Primary Genre	\
0	Drake	Male	Canada	English	Hip-Hop	
1	Taylor Swift	Female	United States	English	Pop	
2	Bad Bunny	Male	United States	Spanish	Reggaeton	
3	The Weeknd	Male	Canada	English	R&B	
4	Justin Bieber	Male	Canada	English	Pop	

	Artist Type	Debut Year	Total Streams (in millions)	\
0	Solo	2006	137492.1	
1	Solo	2006	127861.0	
2	Solo	2013	125899.8	
3	Solo	2009	95703.2	
4	Solo	2009	78104.4	

	Lead Streams (in millions)	Feature Streams (in millions)	\
0	94851.1		42640.9
1	126177.3		1683.8
2	82189.3		43710.5
3	77189.4		18513.8
4	48316.2		29788.2

	Solo Streams (in millions)	% of Solo Streams	\
0	53323.9	38.783246	
1	113421.5	88.706877	
2	48887.0	38.830086	
3	51020.0	53.310652	
4	29638.3	37.947030	

	Collaborative Streams (in millions)	% of Collaborative Streams
0	84168.2	61.216754
1	14439.5	11.293123
2	77012.8	61.169914
3	44683.2	46.689348
4	48466.1	62.052970

```
[4]: # Checking for missing data
spotify.isna().sum()
```

```
[4]: Artist Name      0
     Sex              0
     Country of Origin 0
     Primary Language 0
     Primary Genre     0
     Artist Type       0
     Debut Year        0
     Total Streams (in millions) 0
     Lead Streams (in millions) 0
     Feature Streams (in millions) 0
     Solo Streams (in millions) 0
     % of Solo Streams 0
     Collaborative Streams (in millions) 0
     % of Collaborative Streams 0
     dtype: int64
```

```
[5]: # Fixing Artist Type column name

spotify['Artist Type'] = spotify[' Artist Type']

spotify = spotify.drop(columns=' Artist Type')

# Returning Artist Type to its original place
```

```
spotify = spotify[['Artist Name', 'Sex', 'Country of Origin', 'Primary_
↳Language',
    'Primary Genre', 'Artist Type', 'Debut Year', 'Total Streams (in_
↳millions)',
    'Lead Streams (in millions)', 'Feature Streams (in millions)',
    'Solo Streams (in millions)', '% of Solo Streams',
    'Collaborative Streams (in millions)', '% of Collaborative Streams']]
```

```
[6]: # Checking & previewing dtypes to check wether some column need to be adjusted
spotify.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Artist Name                          500 non-null   object
1   Sex                                  500 non-null   object
2   Country of Origin                    500 non-null   object
3   Primary Language                     500 non-null   object
4   Primary Genre                        500 non-null   object
5   Artist Type                          500 non-null   object
6   Debut Year                           500 non-null   int64
7   Total Streams (in millions)          500 non-null   float64
8   Lead Streams (in millions)           500 non-null   float64
9   Feature Streams (in millions)        500 non-null   float64
10  Solo Streams (in millions)            500 non-null   float64
11  % of Solo Streams                    500 non-null   float64
12  Collaborative Streams (in millions)   500 non-null   float64
13  % of Collaborative Streams            500 non-null   float64
dtypes: float64(7), int64(1), object(6)
memory usage: 54.8+ KB
```

Going to check the str column to check wether they could be turned into categorical column or using feature engineering by collapsing some values into one categories. Except, Artist Name of course.

```
[7]: spotify['Sex'].value_counts()
```

```
[7]: Sex
Male      390
Female     88
Mixed      22
Name: count, dtype: int64
```

```
[8]: spotify['Country of Origin'].value_counts()
```

```
[8]: Country of Origin
      United States      265
      United Kingdom    55
      Mexico            27
      Canada            18
      Colombia          16
      Brazil            15
      India             13
      South Korea       11
      Argentina         9
      Australia         7
      Germany           7
      Spain             6
      Sweden            5
      France            5
      Ireland           4
      Italy             3
      Japan             2
      Scotland          2
      Jamaica           2
      Nigeria           2
      Dominican Republic 2
      Norway            2
      Netherlands       2
      Barbados          1
      Trinidad and Tobago 1
      Panama            1
      Cuba              1
      Morocco           1
      New Zealand        1
      Senegal            1
      Chile              1
      South Africa       1
      Russia             1
      Venezuela          1
      Belgium            1
      DR Congo           1
      Nepal              1
      Pakistan           1
      Guatemala          1
      Austria            1
      Philippines        1
      Denmark            1
      Iceland            1
      Name: count, dtype: int64
```

```
[9]: spotify['Primary Language'].value_counts()
```

```
[9]: Primary Language
      English      340
      Spanish     105
      Portuguese   15
      Korean       11
      Hindi        11
      German        7
      French        4
      Italian       3
      Tamil         2
      Urdu          1
      Punjabi       1
      Name: count, dtype: int64
```

```
[10]: spotify['Primary Genre'].value_counts()
```

```
[10]: Primary Genre
      Hip-Hop      115
      Pop          102
      Rock          60
      Reggaeton    43
      R&B           31
      Latin        30
      EDM          22
      Regional Mexican 19
      K-Pop        11
      Filmi        11
      Country      10
      Alternative   9
      Sertanjeo     9
      Metal         8
      Folk          4
      Soundtrack    3
      Bachata       3
      Soul          3
      Reggae        2
      Afrobeats     2
      Spoken Word   1
      Children's Music 1
      Nu Metal      1
      Name: count, dtype: int64
```

```
[11]: spotify['Artist Type'].value_counts()
```

```
[11]: Artist Type
      Solo      373
      Group    127
```

Name: count, dtype: int64

Sex and Artist Type is definitely better as a categorical group. But Country of Origin, Primary language and Primary genre need grouping in order to lower its cardinality and to improve its statistical power. First i will group Country of Origin into based on its geographical position and market situation.

```
[12]: # Separate each values into a dictionary
region_mapping = {
    # North American Market (Primarily English-speaking)
    'United States': 'North America', 'Canada': 'North America',

    # Latin America & The Caribbean
    'Mexico': 'Latin America', 'Colombia': 'Latin America', 'Barbados': 'Latin_
↪America',
    'Trinidad and Tobago': 'Latin America', 'Cuba': 'Latin America', 'Panama':_
↪'Latin America',
    'Brazil': 'Latin America', 'Jamaica': 'Latin America', 'Argentina': 'Latin_
↪America',
    'Chile': 'Latin America', 'Venezuela': 'Latin America', 'Dominican_
↪Republic': 'Latin America',
    'Guatemala': 'Latin America',

    # European Market
    'United Kingdom': 'Europe', 'Scotland': 'Europe', 'France': 'Europe',
    'Germany': 'Europe', 'Sweden': 'Europe', 'Norway': 'Europe', 'Netherlands':_
↪'Europe',
    'Ireland': 'Europe', 'Spain': 'Europe', 'Russia': 'Europe', 'Italy':_
↪'Europe',
    'Belgium': 'Europe', 'Austria': 'Europe', 'Denmark': 'Europe', 'Iceland':_
↪'Europe',

    # Asian Market
    'South Korea': 'Asia', 'India': 'Asia', 'Japan': 'Asia',
    'Nepal': 'Asia', 'Pakistan': 'Asia', 'Philippines': 'Asia',

    # African Market
    'Senegal': 'Africa', 'Morocco': 'Africa', 'Nigeria': 'Africa',
    'South Africa': 'Africa', 'DR Congo': 'Africa',

    # Oceania Market
    'Australia': 'Oceania', 'New Zealand': 'Oceania'
}

# Apply the mapping using the .map() function
spotify['Region'] = spotify['Country of Origin'].map(region_mapping)
```

```
# Check the results
print("--- New Regional Distribution ---")
print(spotify['Region'].value_counts())
```

--- New Regional Distribution ---

```
Region
North America    283
Europe           96
Latin America    78
Asia             29
Oceania          8
Africa           6
Name: count, dtype: int64
```

Next, I will group Primary Genre into 7 large category and elaborate on each one below:

1. Pop & Electronic

Included: Pop, EDM, K-Pop

The Justification: These genres share massive global crossover appeal and are heavily driven by singles, viral social media trends, and dominant placement on Spotify’s flagship playlists (like Today’s Top Hits). Their streaming curves often look identical: massive initial spikes followed by a steady plateau.

2. Hip-Hop & R&B (Urban/Rhythmic)

Included: Hip-Hop, R&B, Soul

The Justification: These genres share a massive overlapping demographic and dominate the US streaming market. More importantly, artists in these genres heavily collaborate (features and remixes), meaning their stream counts are deeply interconnected through shared listener networks.

3. Rock & Alternative

Included: Rock, Alternative, Metal, Nu Metal

The Justification: Rock and Metal listeners have very distinct platform behaviors. They tend to be album-oriented listeners rather than single-oriented, and they demonstrate high long-term loyalty but lower daily rotational streaming compared to Pop or Hip-Hop.

4. Latin & Regional

Included: Reggaeton, Latin, Regional Mexican, Sertanjeo, Bachata

The Justification: This represents the fastest-growing streaming market in the world. While “Sertanjeo” is Brazilian and “Regional Mexican” is Mexican, they function identically on a macroeconomic scale: massive, highly localized streaming numbers that frequently cross over into Reggaeton collaborations.

5. Country & Folk

Included: Country, Folk

The Justification: These are heavily acoustic, singer-songwriter-driven genres. From a data perspective, they have a highly concentrated domestic audience (primarily North America) and rely heavily on legacy catalog streams rather than just new releases.

6. Media & Soundtrack

Included: Filmi, Soundtrack, Children’s Music

The Justification: Streams in this category are driven by external media (movies, Bollywood films, cartoons) rather than standard artist fandom. A user streaming a “Children’s Music” track is likely a parent playing it on a loop, which creates a completely different data signature than someone streaming a rock band.

7. Global Roots

Included: Afrobeats, Reggae, Spoken Word

The Justification: While distinct culturally, these represent niche or emerging global markets in this specific 500-row dataset. Grouping them preserves their status as non-Western-pop genres without discarding them into an anonymous “Other” bucket.

```
[13]: # Define the domain-aware genre mapping
genre_mapping = {
    # Pop & Electronic
    'Pop': 'Pop & Electronic', 'EDM': 'Pop & Electronic', 'K-Pop': 'Pop &
    ↪Electronic',

    # Hip-Hop & R&B
    'Hip-Hop': 'Hip-Hop & R&B', 'R&B': 'Hip-Hop & R&B', 'Soul': 'Hip-Hop & R&B',

    # Rock & Alternative
    'Rock': 'Rock & Alternative', 'Alternative': 'Rock & Alternative',
    'Metal': 'Rock & Alternative', 'Nu Metal': 'Rock & Alternative',

    # Latin & Regional
    'Reggaeton': 'Latin & Regional', 'Latin': 'Latin & Regional',
    'Regional Mexican': 'Latin & Regional', 'Sertanjeo': 'Latin & Regional',
    'Bachata': 'Latin & Regional',

    # Country & Folk
    'Country': 'Country & Folk', 'Folk': 'Country & Folk',

    # Media & Soundtrack
    'Filmi': 'Media & Soundtrack', 'Soundtrack': 'Media & Soundtrack',
    "Children's Music": 'Media & Soundtrack',

    # Global Roots
    'Afrobeats': 'Global Roots', 'Reggae': 'Global Roots', 'Spoken Word':
    ↪'Global Roots'
}
```



```

# Apply the mapping
spotify['Collapsed Genre'] = spotify['Primary Genre'].map(genre_mapping)

# Check the results
print("--- New Macro-Genre Distribution ---")
print(spotify['Collapsed Genre'].value_counts())

```

--- New Macro-Genre Distribution ---

```

Collapsed Genre
Hip-Hop & R&B      149
Pop & Electronic    135
Latin & Regional    104
Rock & Alternative   78
Media & Soundtrack   15
Country & Folk       14
Global Roots        5
Name: count, dtype: int64

```

Next, I will collapse the value in Primary Language into Other if they are less than 1% of the data. Thus, if the value counts is below 5, the data will be grouped into others

```

[14]: # Define the columns you want to process and the threshold

columns_to_collapse = ['Primary Language']
threshold = 0.01

# Loop through each specified column
for col in columns_to_collapse:

    # Calculate frequencies for the current column
    frequencies = spotify[col].value_counts(normalize=True)

    # Identify categories that fall below the 1% threshold
    rare_categories = frequencies[frequencies < threshold].index

    # Replace the rare categories with 'Other'
    spotify[col] = spotify[col].replace(rare_categories, 'Other')

# Check the results
print("--- New Language Distribution ---")
print(spotify['Primary Language'].value_counts())

```

--- New Language Distribution ---

```

Primary Language
English      340
Spanish      105
Portuguese    15
Korean        11

```

```
Hindi          11
Other          11
German         7
Name: count, dtype: int64
```

Finally, turn all of those column into categorical dtypes

```
[15]: # Changing into categorical dtype
spotify['Sex'] = spotify['Sex'].astype('category')
spotify['Primary Language'] = spotify['Primary Language'].astype('category')
spotify['Genre'] = spotify['Primary Genre'].astype('category')
spotify['Region'] = spotify['Country of Origin'].astype('category')
spotify['Artist Type'] = spotify['Artist Type'].astype('category')

# Checking
spotify.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Artist Name                          500 non-null    object
1   Sex                                  500 non-null    category
2   Country of Origin                    500 non-null    object
3   Primary Language                     500 non-null    category
4   Primary Genre                        500 non-null    object
5   Artist Type                          500 non-null    category
6   Debut Year                           500 non-null    int64
7   Total Streams (in millions)          500 non-null    float64
8   Lead Streams (in millions)           500 non-null    float64
9   Feature Streams (in millions)        500 non-null    float64
10  Solo Streams (in millions)           500 non-null    float64
11  % of Solo Streams                    500 non-null    float64
12  Collaborative Streams (in millions)  500 non-null    float64
13  % of Collaborative Streams           500 non-null    float64
14  Region                              500 non-null    category
15  Collapsed Genre                      500 non-null    object
16  Genre                                500 non-null    category
dtypes: category(5), float64(7), int64(1), object(4)
memory usage: 52.1+ KB
```

```
[29]: #Reordering the data into final df to be processed in the next step
sub_spotify = spotify[['Artist Name', 'Sex', 'Region', 'Primary Language',
↳ 'Collapsed Genre', 'Artist Type', 'Debut Year', 'Total Streams (in_
↳ millions)',\
                                'Lead Streams (in millions)', 'Feature Streams (in_
↳ millions)', 'Solo Streams (in millions)', '% of Solo Streams',\
```

```

        'Collaborative Streams (in millions)', '% of_
↳Collaborative Streams']]

# Saving externally for documentation
sub_spotify.to_csv('C:\\Users\\ih-san\\Data Analysis\\Most Streamed Artists on_
↳Spotify\\Data\\Sub_spotify.csv', index=False)

# Checking
sub_spotify.head()

```

```

[29]:      Artist Name      Sex      Region Primary Language  Collapsed Genre \
0      Drake      Male      Canada      English  Hip-Hop & R&B
1  Taylor Swift  Female  United States      English  Pop & Electronic
2    Bad Bunny      Male  United States      Spanish  Latin & Regional
3    The Weeknd      Male      Canada      English  Hip-Hop & R&B
4  Justin Bieber      Male      Canada      English  Pop & Electronic

```

```

      Artist Type  Debut Year  Total Streams (in millions) \
0      Solo      2006      137492.1
1      Solo      2006      127861.0
2      Solo      2013      125899.8
3      Solo      2009      95703.2
4      Solo      2009      78104.4

```

```

      Lead Streams (in millions)  Feature Streams (in millions) \
0      94851.1      42640.9
1      126177.3      1683.8
2      82189.3      43710.5
3      77189.4      18513.8
4      48316.2      29788.2

```

```

      Solo Streams (in millions)  % of Solo Streams \
0      53323.9      38.783246
1      113421.5      88.706877
2      48887.0      38.830086
3      51020.0      53.310652
4      29638.3      37.947030

```

```

      Collaborative Streams (in millions)  % of Collaborative Streams
0      84168.2      61.216754
1      14439.5      11.293123
2      77012.8      61.169914
3      44683.2      46.689348
4      48466.1      62.052970

```

Much better!

0.2 EDA Phase

```
[17]: # Apply the theme
palette = sns.color_palette(palette='Set2')
sns.set_theme(style="whitegrid", palette=palette, font="sans-serif")
sns.palplot(palette)
plt.show()
```



```
[73]: # Making pie chart to visualize the artist sex distribution

# Preparing the value counts
class_counts = sub_spotify['Sex'].value_counts()

labels = class_counts.index
values = class_counts.values

# Create a custom formatting function to turn the value into percentage
def autopct(pct):
    # Calculate the raw count from the percentage
    total = sum(values)
    val = int(round(pct * total / 100.0))
    return f'{pct:.1f}%'

# Plot the pie chart using the custom function
plt.figure(figsize=(6, 6))
plt.pie(
    values,
    autopct=autopct, #showing percentage to the pie chart
    startangle=90,
    textprops={'fontsize': 10}
)

# Customizing the legend
plt.legend(labels,
           title="Categories",
           loc="center left",
           bbox_to_anchor=(1, 0.5), #putting the legend on the right side of
↳ the chart
           fontsize=10,
```

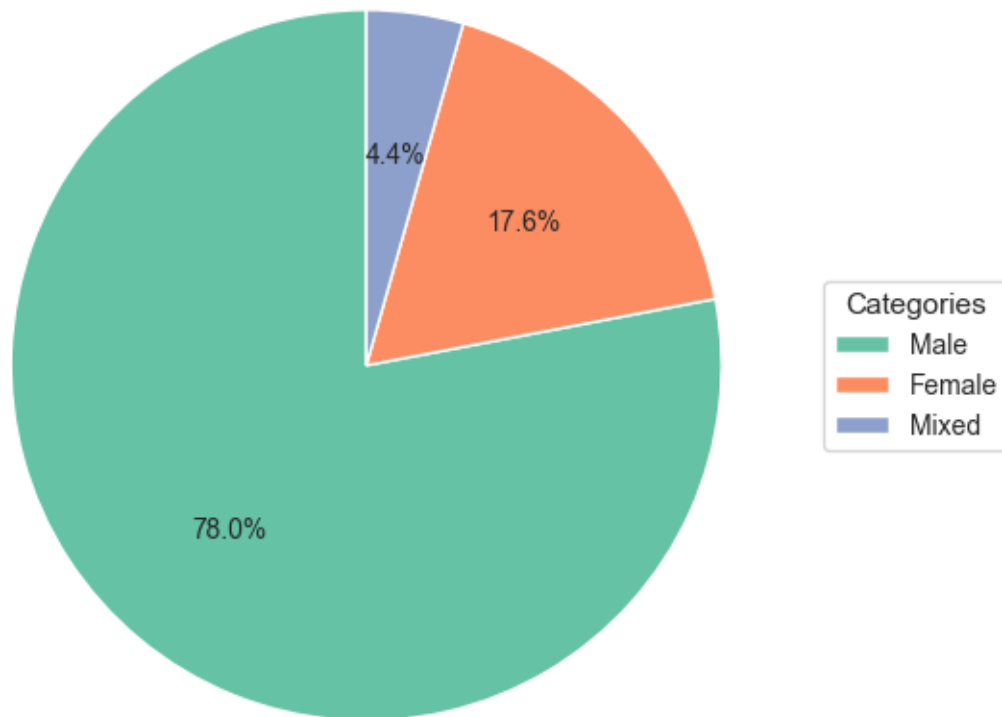
```

        title_fontsize=11)

plt.title('Spotify Artist Sex Distribution', fontweight="bold")
plt.tight_layout()
plt.savefig('C:\\Users\\ih-san\\Data Analysis\\Most Streamed Artists on Spotify\\Visuals\\Spotify Artist Sex Distribution.png')
plt.show()

```

Spotify Artist Sex Distribution



```

[74]: # Making pie chart to visualize the artist type distribution

# Preparing the value counts
class_counts = sub_spotify['Artist Type'].value_counts()

labels = class_counts.index
values = class_counts.values

# Create a custom formatting function to turn the value into percentage

```

```

def autopct(pct):
    # Calculate the raw count from the percentage
    total = sum(values)
    val = int(round(pct * total / 100.0))
    return f'{pct:.1f}%'

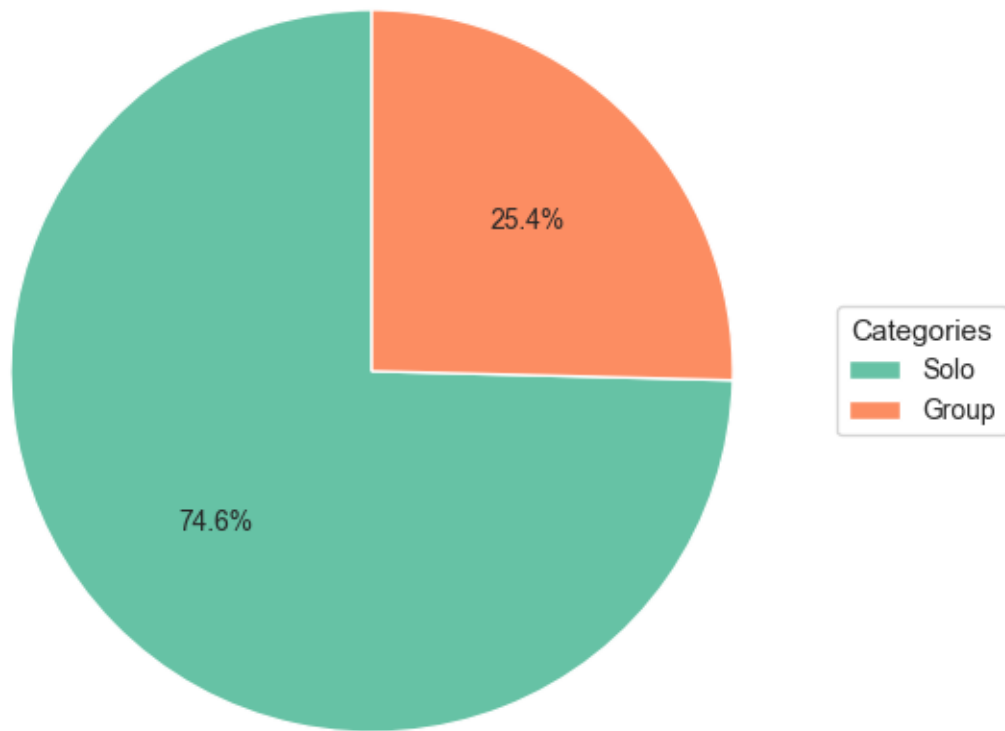
# Plot the pie chart using the custom function
plt.figure(figsize=(6, 6))
plt.pie(
    values,
    autopct=autopct, #showing percentage to the pie chart
    startangle=90,
    textprops={'fontsize': 10}
)

# Customizing the legend
plt.legend(labels,
           title="Categories",
           loc="center left",
           bbox_to_anchor=(1, 0.5), #putting the legend on the right side of
           ↪the chart
           fontsize=10,
           title_fontsize=11)

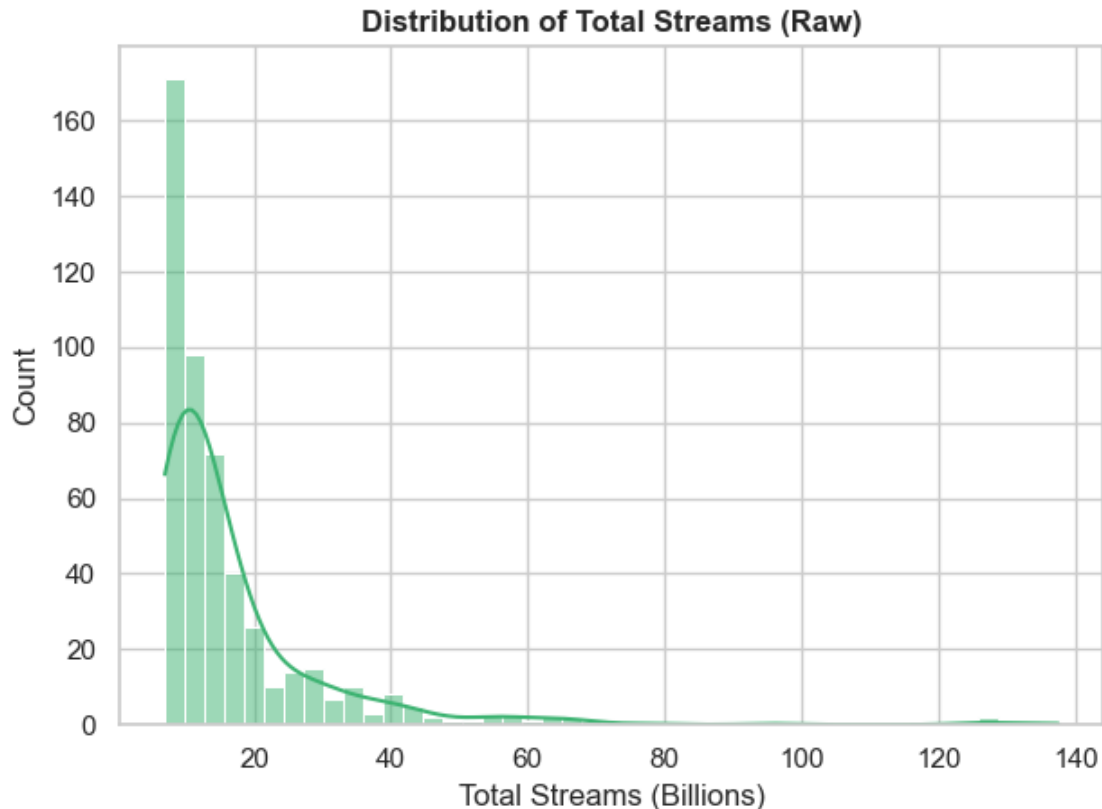
plt.title('Spotify Artist Type Distribution', fontweight="bold")
plt.tight_layout()
plt.savefig('C:\\Users\\ih-san\\Data Analysis\\Most Streamed Artists on
           ↪Spotify\\Visuals\\Spotify Artist Type Distribution.png')
plt.show()

```

Spotify Artist Type Distribution



```
[75]: # Distribution of Total Streams
sns.histplot(sub_spotify['Total Streams (in millions)'], kde=True,
             color='mediumseagreen')
plt.title('Distribution of Total Streams (Raw)', fontweight="bold")
plt.xticks([20000, 40000, 60000, 80000, 100000, 120000, 140000], ['20', '40',
             '60', '80', '100', '120', '140'])
plt.xlabel('Total Streams (Billions)')
plt.tight_layout()
plt.savefig('C:\\Users\\ih-san\\Data Analysis\\Most Streamed Artists on
             Spotify\\Visuals\\Total Streams (Billions).png')
plt.show()
```



It seems like the total stream data is skewed to the right. The long tail on the right suggests that there are a few outliers with very high values. A logarithmic scale can help to compress these large values, making the distribution more symmetric and easier to visualize. But it would also mean the data would be harder to interpret and internalized.

```
[76]: # Debut Year vs Total Streams by Sex / Artist Type
plt.figure(figsize=(10, 6))
sns.scatterplot(data=sub_spotify, x='Debut Year', y='Total Streams (in_
    ↳ millions)', hue='Sex', style='Artist Type', alpha=0.7, s=70)
plt.title('Debut Year vs. Total Streams by Sex and Artist Type',
    ↳ fontweight="bold")
plt.xlabel('Debut Year')
plt.ylabel('Total Streams (Billions)')
plt.yticks([20000, 40000, 60000, 80000, 100000, 120000, 140000], ['20', '40',
    ↳ '60', '80', '100', '120', '140'])
plt.tight_layout()
plt.savefig('C:\\Users\\ih-san\\Data Analysis\\Most Streamed Artists on_
    ↳ Spotify\\Visuals\\Debut Year vs. Total Streams by Sex and Artist Type.png')
plt.show()
```

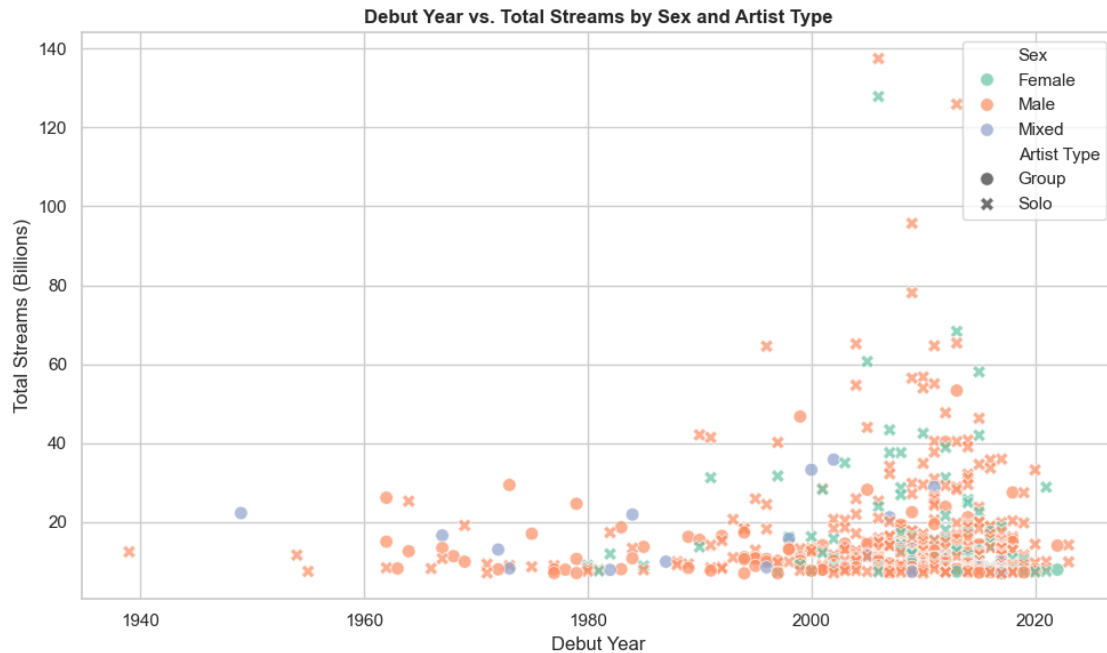
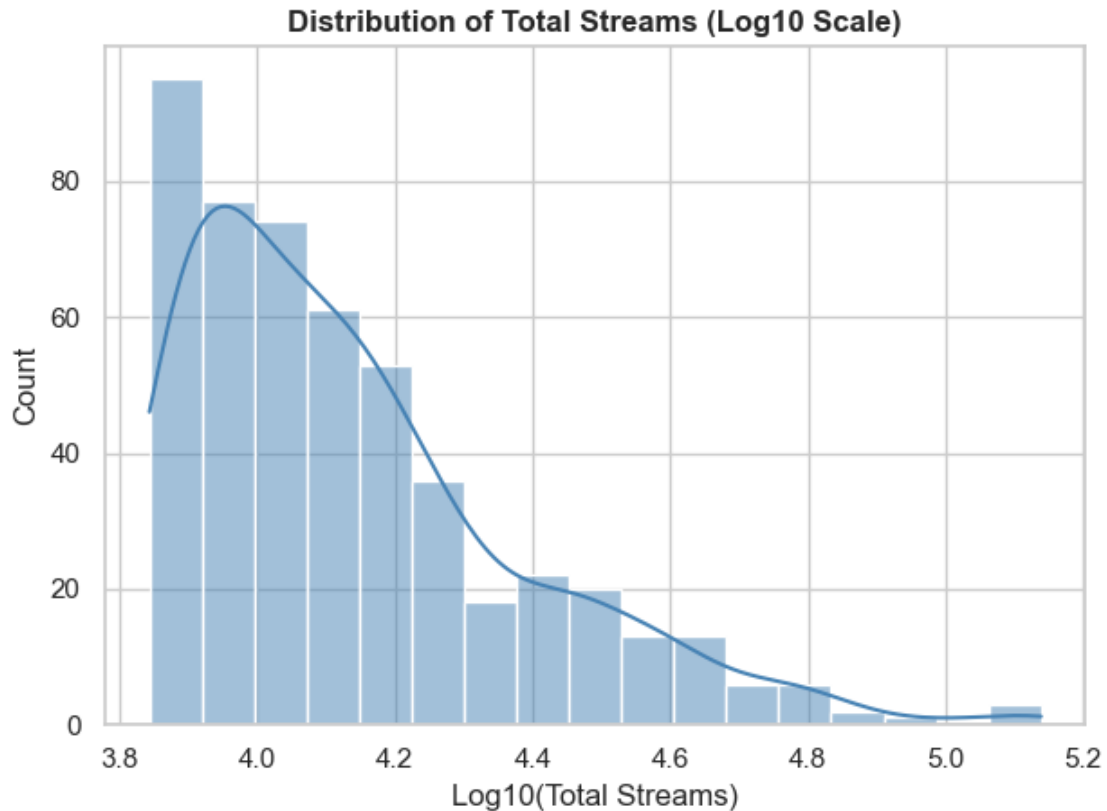
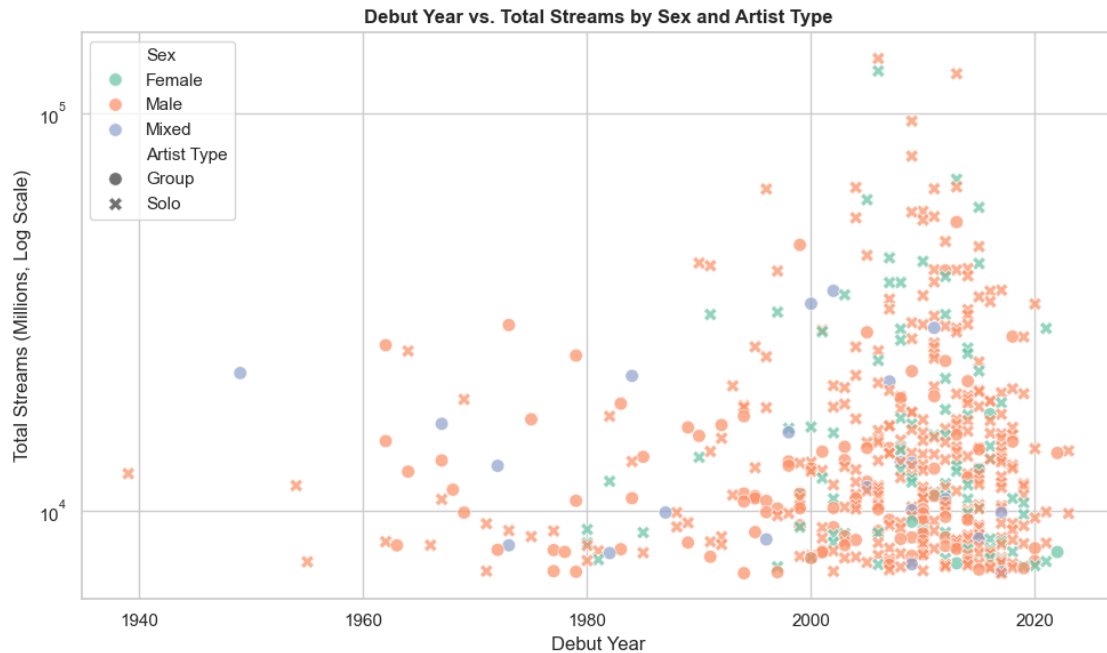



Chart above is basically summarizing the other three charts

```
[77]: sns.histplot(np.log10(sub_spotify['Total Streams (in millions)']), kde=True,
    color='steelblue')
plt.title('Distribution of Total Streams (Log10 Scale)', fontweight="bold")
plt.xlabel('Log10(Total Streams)')
plt.tight_layout()
plt.savefig('C:\\Users\\ih-san\\Data Analysis\\Most Streamed Artists on
    Spotify\\Visuals\\Distribution of Total Streams (Log10 Scale).png')
plt.show()
```

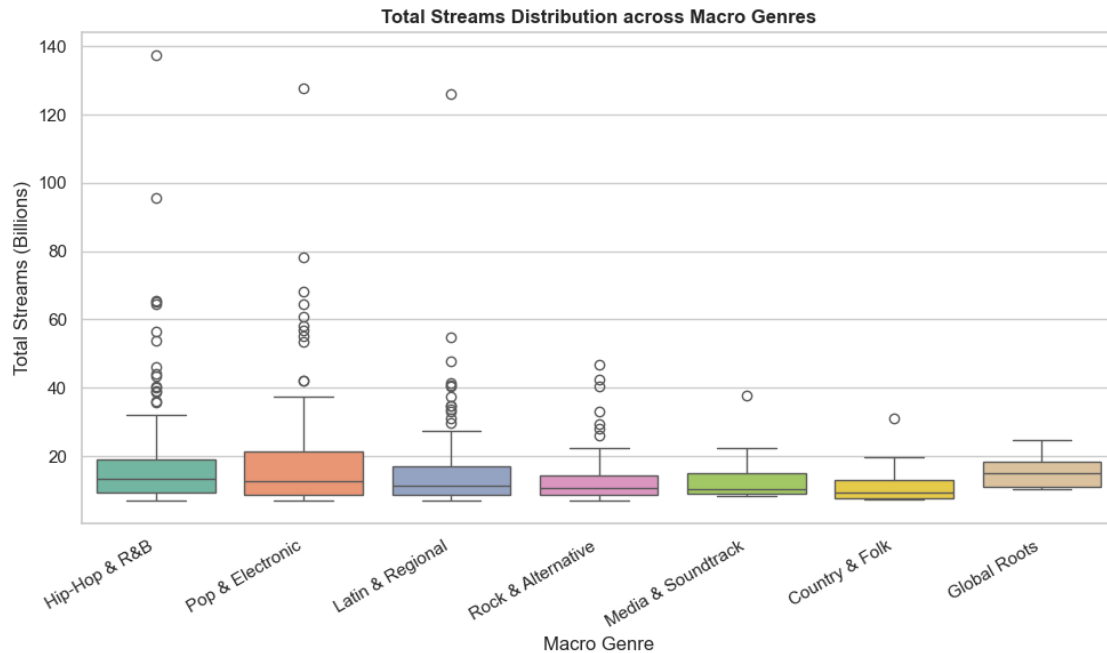


```
[78]: # Debut Year vs Total Streams by Sex / Artist Type (scaled)
plt.figure(figsize=(10, 6))
sns.scatterplot(data=sub_spotify, x='Debut Year', y='Total Streams (in_
    ↪millions)', hue='Sex', style='Artist Type', alpha=0.7, s=70)
plt.title('Debut Year vs. Total Streams by Sex and Artist Type',
    ↪fontweight="bold")
plt.yscale('log')
plt.xlabel('Debut Year')
plt.ylabel('Total Streams (Millions, Log Scale)')
plt.tight_layout()
plt.savefig('C:\\Users\\ih-san\\Data Analysis\\Most Streamed Artists on_
    ↪Spotify\\Visuals\\Total Streams (Millions, Log Scale).png')
plt.show()
```



Yup, now the data is also easier to view. And it seems like the number of debuting male solo artist also increased exponentially over the years. Which is why male and solo artist is dominant in this dataset.

```
[79]: plt.figure(figsize=(10, 6))
sns.boxplot(data=sub_spotify, x='Collapsed Genre', hue='Collapsed Genre',
            y='Total Streams (in millions)')
plt.xticks(rotation=30, ha='right')
plt.title('Total Streams Distribution across Macro Genres', fontweight="bold")
plt.ylabel('Total Streams (Billions)')
plt.yticks([20000, 40000, 60000, 80000, 100000, 120000, 140000], ['20', '40',
    '60', '80', '100', '120', '140'])
plt.xlabel('Macro Genre')
plt.tight_layout()
plt.savefig('C:\\Users\\ihshan\\Data Analysis\\Most Streamed Artists on
    Spotify\\Visuals\\Total Streams Distribution across Macro Genres.png')
plt.show()
```



```
[25]: # 2. Define the IQR Outlier formula
def count_outliers(series):
    Q1 = series.quantile(0.25)
    Q3 = series.quantile(0.75)
    IQR = Q3 - Q1

    # Calculate the bounds
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Return the count of values outside the bounds
    return ((series < lower_bound) | (series > upper_bound)).sum()

# 3. Apply the function to each genre
outliers = sub_spotify.groupby('Collapsed Genre')['Total Streams (in millions)'].apply(count_outliers)
print("--- Number of outlier ---")
print(outliers)
```

```
--- Number of outlier ---
Collapsed Genre
Country & Folk      1
Global Roots       0
Hip-Hop & R&B      16
Latin & Regional    13
Media & Soundtrack  1
```

```
Pop & Electronic      11
Rock & Alternative    7
Name: Total Streams (in millions), dtype: int64
```

```
[80]: # Making pie chart to visualize the artist sex distribution

# Preparing the value counts
class_counts = sub_spotify['Collapsed Genre'].value_counts()

labels = class_counts.index
values = class_counts.values

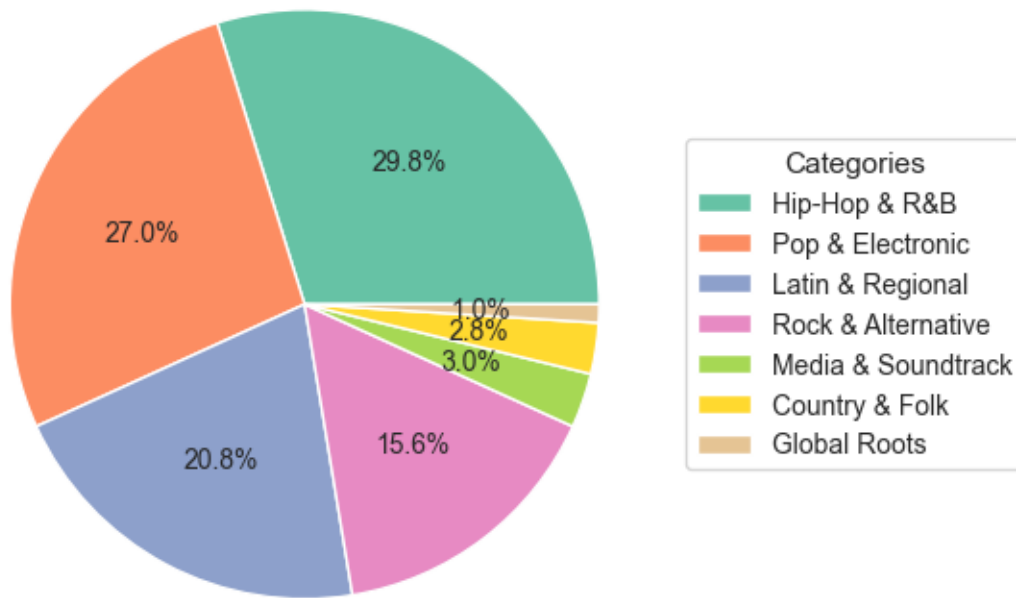
# Create a custom formatting function to turn the value into percentage
def autopct(pct):
    # Calculate the raw count from the percentage
    total = sum(values)
    val = int(round(pct * total / 100.0))
    return f'{pct:.1f}%'

# Plot the pie chart using the custom function
plt.figure(figsize=(6, 6))
plt.pie(
    values,
    autopct=autopct, # showing percentage to the pie chart
    startangle=0,
    textprops={'fontsize': 10}
)

# Customizing the legend
plt.legend(labels,
           title="Categories",
           loc="center left",
           bbox_to_anchor=(1, 0.5), # putting the legend on the right side of ↵
           ↵the chart
           fontsize=10,
           title_fontsize=11)

plt.title('Spotify Artist Genre Distribution', fontweight="bold")
plt.tight_layout()
plt.savefig('C:\\Users\\ih-san\\Data Analysis\\Most Streamed Artists on ↵
           ↵Spotify\\Visuals\\Spotify Artist Genre Distribution.png')
plt.show()
```

Spotify Artist Genre Distribution



Since genre with top 3 highest number of outlier is also the top 3 in the overall data distribution, it seems normal. Also another thing to note, all of the outliers are upper outlier. Which mean all of them are the top performers in their own genre.

Here are all of the outlier in the dataset based on Total Streams (in millions)

```
[69]: # Calculate Q1 and Q3
Q1 = sub_spotify['Total Streams (in millions)'].quantile(0.25)
Q3 = sub_spotify['Total Streams (in millions)'].quantile(0.75)

# Calculate IQR
IQR = Q3 - Q1

# Define lower and upper bounds for outliers
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# subsetting outliers
outliers = sub_spotify[(sub_spotify['Total Streams (in millions)'] <
    ↳ lower_bound) | (sub_spotify['Total Streams (in millions)'] > upper_bound)]
outliers.to_csv('C:\\Users\\ihshan\\Data Analysis\\Most Streamed Artists on
    ↳ Spotify\\Data\\outliers.csv', index=False)
```

```
print("Outliers:")
outliers.head(48)
```

Outliers:

```
[69]:
```

	Artist Name	Sex	Region	Primary Language	\
0	Drake	Male	Canada	English	
1	Taylor Swift	Female	United States	English	
2	Bad Bunny	Male	United States	Spanish	
3	The Weeknd	Male	Canada	English	
4	Justin Bieber	Male	Canada	English	
5	Ariana Grande	Female	United States	English	
6	Travis Scott	Male	United States	English	
7	Kanye West	Male	United States	English	
8	Ed Sheeran	Male	United Kingdom	English	
9	Eminem	Male	United States	English	
10	Rihanna	Female	Barbados	English	
11	Billie Eilish	Female	United States	English	
12	Bruno Mars	Male	United States	English	
13	Kendrick Lamar	Male	United States	English	
14	Post Malone	Male	United States	English	
15	J Balvin	Male	Colombia	Spanish	
16	Future	Male	United States	English	
17	BTS	Male	South Korea	Korean	
18	Ozuna	Male	United States	Spanish	
19	Coldplay	Male	United Kingdom	English	
20	Juice WRLD	Male	United States	English	
21	Chris Brown	Male	United States	English	
22	Nicki Minaj	Female	Trinidad and Tobago	English	
23	Lana Del Rey	Female	United States	English	
24	David Guetta	Male	France	English	
25	Dua Lipa	Female	United Kingdom	English	
26	Daddy Yankee	Male	United States	Spanish	
27	Rauw Alejandro	Male	United States	Spanish	
28	Anuel AA	Male	United States	Spanish	
29	XXXTENTACION	Male	United States	English	
30	Imagine Dragons	Male	United States	English	
31	Lil Wayne	Male	United States	English	
32	21 Savage	Male	United Kingdom	English	
33	SZA	Female	United States	English	
34	Arijit Singh	Male	India	Hindi	
35	Lady Gaga	Female	United States	English	
36	KAROL G	Female	Colombia	Spanish	
37	Lil Baby	Male	United States	English	
38	Maroon 5	Mixed	United States	English	
39	Khalid	Male	United States	English	

40	Beyoncé	Female	United States	English
41	Maluma	Male	Colombia	Spanish
42	Feid	Male	Colombia	Spanish
43	Calvin Harris	Male	Scotland	English
44	Myke Towers	Male	United States	Spanish
45	Linkin Park	Mixed	United States	English
46	Peso Pluma	Male	Mexico	Spanish
47	J. Cole	Male	United States	English

	Collapsed Genre	Artist	Type	Debut	Year	Total Streams (in millions)	\
0	Hip-Hop & R&B		Solo		2006	137492.1	
1	Pop & Electronic		Solo		2006	127861.0	
2	Latin & Regional		Solo		2013	125899.8	
3	Hip-Hop & R&B		Solo		2009	95703.2	
4	Pop & Electronic		Solo		2009	78104.4	
5	Pop & Electronic		Solo		2013	68298.9	
6	Hip-Hop & R&B		Solo		2013	65349.7	
7	Hip-Hop & R&B		Solo		2004	65142.9	
8	Pop & Electronic		Solo		2011	64656.9	
9	Hip-Hop & R&B		Solo		1996	64549.0	
10	Pop & Electronic		Solo		2005	60670.7	
11	Pop & Electronic		Solo		2015	58049.5	
12	Pop & Electronic		Solo		2010	56727.1	
13	Hip-Hop & R&B		Solo		2009	56459.5	
14	Pop & Electronic		Solo		2011	55028.3	
15	Latin & Regional		Solo		2004	54649.2	
16	Hip-Hop & R&B		Solo		2010	53951.5	
17	Pop & Electronic		Group		2013	53308.9	
18	Latin & Regional		Solo		2012	47649.2	
19	Rock & Alternative		Group		1999	46720.4	
20	Hip-Hop & R&B		Solo		2015	46276.4	
21	Hip-Hop & R&B		Solo		2005	43963.8	
22	Hip-Hop & R&B		Solo		2007	43330.3	
23	Rock & Alternative		Solo		2010	42464.1	
24	Pop & Electronic		Solo		1990	42059.5	
25	Pop & Electronic		Solo		2015	41944.1	
26	Latin & Regional		Solo		1991	41405.4	
27	Latin & Regional		Solo		2014	40738.6	
28	Latin & Regional		Solo		2011	40453.3	
29	Hip-Hop & R&B		Solo		2013	40411.3	
30	Rock & Alternative		Group		2012	40279.5	
31	Hip-Hop & R&B		Solo		1997	40148.2	
32	Hip-Hop & R&B		Solo		2014	39024.3	
33	Hip-Hop & R&B		Solo		2012	38879.6	
34	Media & Soundtrack		Solo		2011	37687.8	
35	Pop & Electronic		Solo		2008	37565.4	
36	Latin & Regional		Solo		2007	37549.7	

37	Hip-Hop & R&B	Solo	2017	35931.9
38	Pop & Electronic	Group	2002	35794.1
39	Hip-Hop & R&B	Solo	2016	35727.2
40	Pop & Electronic	Solo	2003	34975.2
41	Latin & Regional	Solo	2010	34841.0
42	Latin & Regional	Solo	2015	34579.1
43	Pop & Electronic	Solo	2007	34129.7
44	Latin & Regional	Solo	2016	33620.7
45	Rock & Alternative	Group	2000	33224.7
46	Latin & Regional	Solo	2020	33179.8
47	Hip-Hop & R&B	Solo	2007	32200.9

	Lead Streams (in millions)	Feature Streams (in millions) \
0	94851.1	42640.9
1	126177.3	1683.8
2	82189.3	43710.5
3	77189.4	18513.8
4	48316.2	29788.2
5	57097.5	11201.4
6	35338.7	30027.0
7	46558.4	18584.5
8	57704.0	6967.8
9	54466.8	10100.7
10	44824.4	15846.4
11	57160.1	889.4
12	40714.0	16013.1
13	40752.2	15725.0
14	48805.6	6235.6
15	25333.3	29334.0
16	27879.0	26072.5
17	47969.9	5339.0
18	20247.1	27418.9
19	42837.1	3901.3
20	34132.1	12155.1
21	24455.7	19523.9
22	16316.4	27027.9
23	38477.3	3986.8
24	30750.8	11325.3
25	28269.7	13687.6
26	19772.1	21650.0
27	24898.0	15857.7
28	18355.6	22118.9
29	34419.9	5998.9
30	38006.7	2283.2
31	15870.4	24279.9
32	15878.0	23155.6
33	25267.5	13627.2

34	13330.4	24384.8
35	37009.7	572.0
36	29603.8	7961.5
37	18773.5	17171.5
38	35585.8	222.6
39	18499.9	17234.4
40	29662.8	5325.3
41	19449.8	15404.7
42	18725.6	15774.1
43	31846.0	2298.3
44	14022.5	19614.7
45	31426.6	1798.1
46	18699.1	14501.2
47	24334.7	7876.1

	Solo Streams (in millions)	% of Solo Streams \
0	53323.9	38.783246
1	113421.5	88.706877
2	48887.0	38.830086
3	51020.0	53.310652
4	29638.3	37.947030
5	41615.5	60.931435
6	25309.9	38.729941
7	28473.0	43.708524
8	46700.0	72.227403
9	32940.0	51.031000
10	30261.8	49.878772
11	51567.9	88.834357
12	33618.1	59.262857
13	23403.5	41.451837
14	25419.4	46.193322
15	7844.5	14.354281
16	12647.5	23.442351
17	44114.4	82.752411
18	8621.1	18.092854
19	38535.2	82.480458
20	26675.7	57.644285
21	12219.3	27.794003
22	9618.2	22.197400
23	34578.4	81.429725
24	406.8	0.967201
25	23133.7	55.153645
26	10692.8	25.824651
27	10432.5	25.608391
28	6177.0	15.269459
29	28138.4	69.630029
30	33880.4	84.113259

31	4050.9	10.089867
32	7883.1	20.200490
33	19804.6	50.938281
34	4537.4	12.039440
35	23546.2	62.680552
36	14790.4	39.388863
37	8928.8	24.849229
38	23312.5	65.129449
39	12520.6	35.045008
40	21863.9	62.512580
41	11616.3	33.340891
42	9877.3	28.564364
43	5998.8	17.576480
44	9406.6	27.978597
45	30151.2	90.749352
46	3938.3	11.869571
47	18999.5	59.003009

	Collaborative Streams (in millions)	% of Collaborative Streams
0	84168.2	61.216754
1	14439.5	11.293123
2	77012.8	61.169914
3	44683.2	46.689348
4	48466.1	62.052970
5	26683.4	39.068565
6	40039.8	61.270059
7	36669.9	56.291476
8	17956.9	27.772597
9	31609.0	48.969000
10	30408.9	50.121228
11	6481.6	11.165643
12	23109.0	40.737143
13	33056.0	58.548163
14	29608.9	53.806678
15	46804.7	85.645719
16	41304.0	76.557649
17	9194.5	17.247589
18	39028.1	81.907146
19	8185.2	17.519542
20	19600.7	42.355715
21	31744.5	72.205997
22	33712.1	77.802600
23	7885.7	18.570275
24	41652.7	99.032799
25	18810.4	44.846355
26	30712.6	74.175349
27	30306.1	74.391609

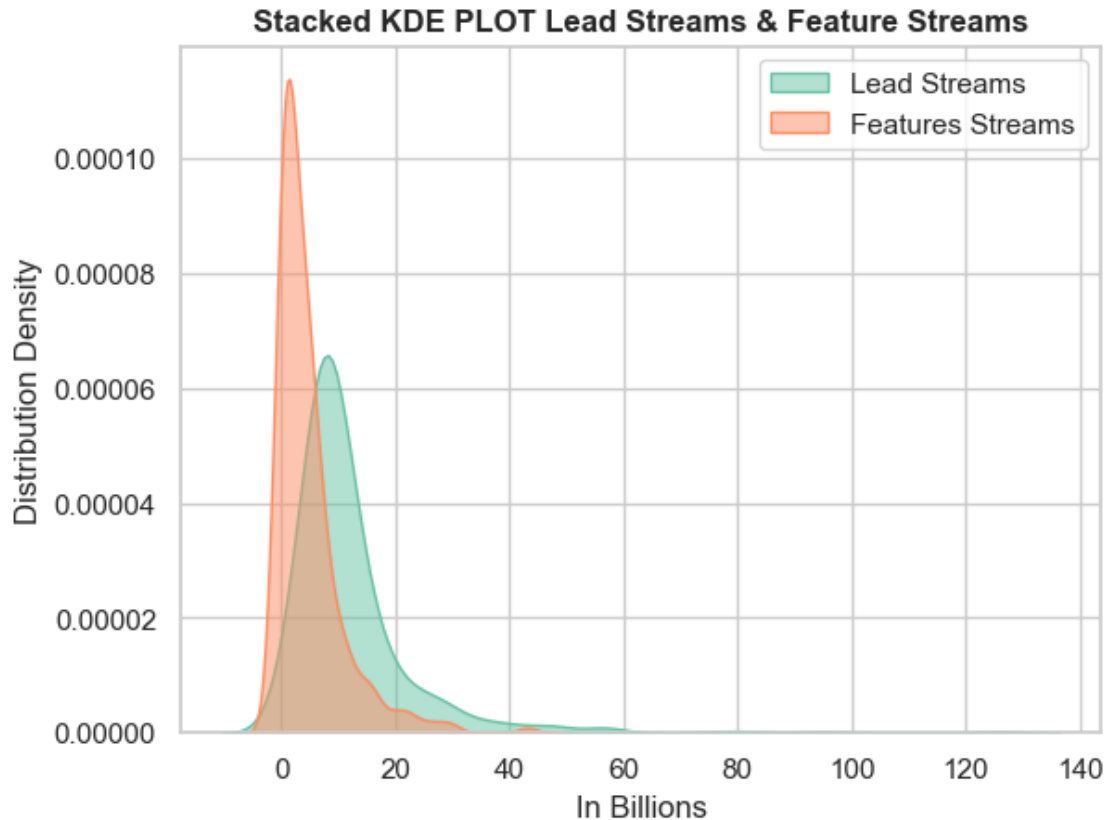
28	34276.3	84.730541
29	12272.9	30.369971
30	6399.1	15.886741
31	36097.3	89.910133
32	31141.2	79.799510
33	19075.0	49.061719
34	33150.4	87.960560
35	14019.2	37.319448
36	22759.3	60.611137
37	27003.1	75.150771
38	12481.6	34.870551
39	23206.6	64.954992
40	13111.3	37.487420
41	23224.7	66.659109
42	24701.8	71.435636
43	28130.9	82.423520
44	24214.1	72.021403
45	3073.5	9.250648
46	29241.5	88.130429
47	13201.4	40.996991

I think visualizing the categorical data is already enough to learn about this dataset. Now, it is time to dig more into the numerical side.

I will make KDE Plot to learn about lead streams, feature streams, solo streams, and collaborative streams. I will separate it into two graph, KDE plot containing lead streams & feature streams, and solo streams & collaborative streams. The reason will be explained on the graph explanation.

```
[81]: # Making KDE Plot containing Lead Streams & Feature Streams
sns.kdeplot(data=sub_spotify, x="Lead Streams (in millions)", fill=True,
            label="Lead Streams", alpha=0.5)
sns.kdeplot(data=sub_spotify, x="Feature Streams (in millions)", fill=True,
            label="Features Streams", alpha=0.5)

# Adjusting the plot
plt.xlabel("In Billions")
plt.xticks([0, 20000, 40000, 60000, 80000, 100000, 120000, 140000], ['0', '20',
            '40', '60', '80', '100', '120', '140'])
plt.ylabel("Distribution Density")
plt.title("Stacked KDE PLOT Lead Streams & Feature Streams", fontweight="bold")
plt.legend()
plt.savefig('C:\\Users\\ihshan\\Data Analysis\\Most Streamed Artists on
            Spotify\\Visuals\\Stacked KDE PLOT Lead Streams & Feature Streams.png')
plt.show()
```



First of all, here is the definition: > Lead Streams: Any track where the artist is the main/primary artist (e.g., Kendrick Lamar's own songs).

Feature Streams: Any track where the artist is a guest on someone else's song (e.g., Kendrick Lamar feature on a A\$AP ROCKY song).

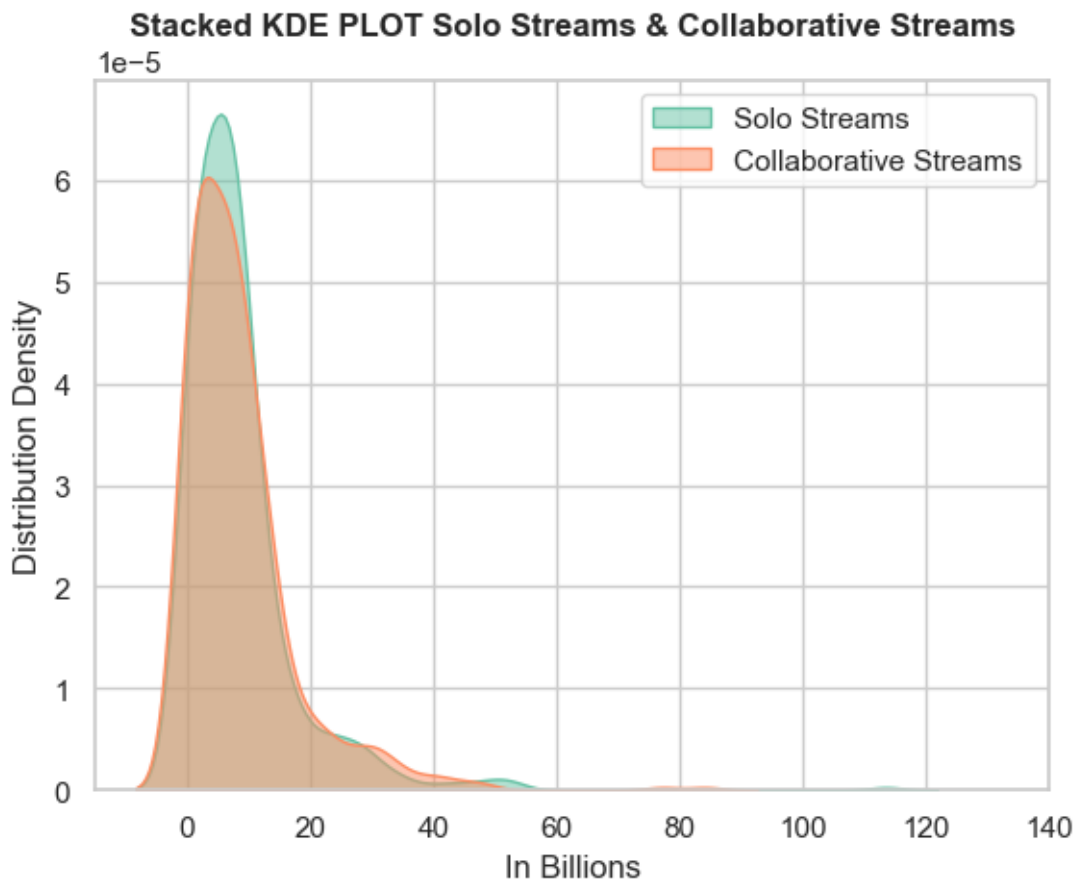
Based on the definition, it is normal that lead streams is more skewed to the right and have lower density. Which mean, Lead streams tend to have higher number and the number is less spread out when compared to feature streams. It is also make sense, since most artist discography are indeed filled with their own song.

I also think it is quite logical to compare this two data, because they are the opposite of each other. Thus it is make sense to find the Contrast between the two.

```
[82]: # Making KDE Plot containing Lead Streams & Feature Streams
sns.kdeplot(data=sub_spotify, x="Solo Streams (in millions)", fill=True,
            label="Solo Streams", alpha=0.5)
sns.kdeplot(data=sub_spotify, x="Collaborative Streams (in millions)",
            fill=True, label="Collaborative Streams", alpha=0.5)

# Adjusting the plot
plt.xlabel("In Billions")
```

```
plt.xticks([0, 20000, 40000, 60000, 80000, 100000, 120000, 140000], ['0', '20', '40', '60', '80', '100', '120', '140'])
plt.ylabel("Distribution Density")
plt.title("Stacked KDE PLOT Solo Streams & Collaborative Streams",
fontweight="bold")
plt.legend()
plt.savefig('C:\\Users\\ihshan\\Data Analysis\\Most Streamed Artists on Spotify\\Visuals\\Stacked KDE PLOT Solo Streams & Collaborative Streams.png')
plt.show()
```



Like before, here is the definition: > Solo Streams: Tracks where the artist is 100% alone. No guests, no co-leads.

Collaborative Streams: total streams accumulated while collaborating with other artists

In this case, collaborative streams is actually Feature Streams + Lead Tracks that have a guest. Thus, that is probably why the distribution is closer to the solo streams. Also, it could probably be seen that today collaboration is more common to cross-pollinate fanbases and boost streams.

0.3 MODELLING

[]: